Project Proposal

CS 6140

Integration of continuous memory systems in revised deep learning models

Group name: torch

Group project proposal by:

- Preethi Shyamsundar - shyamsundar.p@northeastern.edu
  - NUID: 002500876
- Nida Esen - esen.n@northeastern.edu
  - NUID: 002594922
- Tiago Monteiro – monteiro.t@northeastern.edu
  - NUID: 003151399

Contents

Problem Definition 3
Project Scope 4
Methodology and Timeline 4
Until 8 of February 4
Until 15 of February 4
Until 5 of march 5
Until 15 of march 5
Division of the work 6
Preethi work responsibilities - 33% 6
Nida work responsibilities - 33% 7
Tiago's work responsibilities - 34% 8
Conclusion 9
References 9

Problem Definition

RNNs and their variants, like GRU and LSTM, have long been widely used in language tasks and in time series data.

In 2017, due to the transformer neural network architecture, the long-standing issue of sequential processing of data and lack of complete attention was solved.[1]

Nevertheless, due to quadratic time complexity, alternative models like Mamba and new variants like xLSTM, tested in the long-range arena benchmark, proved in certain tasks to beat transformer-based models.[2][3]

An assumption of all RNNs is that the hidden memory is static in relation to the tokens the model parses. In other words, the weights related to the hidden units of an RNN model all learn at the same frequency of the incoming sequence data.

However, in a study by Google, it was presented that machine learning is just a field where different optimization processes happen (sequential, parallel, and nested). In addition, it is possible to create a continuous memory spectrum to make the model's memory better.[4]

This project seeks to understand, based on the idea of continuous memory spectrum in RNNs, if it and gate mixes and other possible architectural combinations like CMS augmented Mamba, vision models, and residual connections (ensuring a strong effect of backpropagation) can lead to better results. [5]

Project Scope

Here in this chapter, we define the project scope by explaining in detail the methodology we are going to employ as well as the timelines we hope to achieve.

In addition, all code will be developed in Google Colab Pro so that we have access to GPUs.

Methodology and Timeline

Until 8 of February

- Understand all the mathematics of the continuous memory system concept in the paper: "**Nested Learning: The Illusion of Deep Learning Architectures**"
- Study and understand the Long-Range area dataset benchmark based on the article: "**Long Range Arena: A Benchmark for Efficient Transformers**"
- Understand how to use the Optuna Python library to guarantee optimal baselines and optimal custom model creation
- Understand how to use PyTorch Lightning to know how to create new models easily.
- Study in depth Mamba, LSTM, GRU, xLSTM, and other gating mechanisms to understand how and why they were used. From the philosophy as to why they were used, we will create our own sequence models.
- With PyTorch, learn to implement strach RNN, LSTM, and GRU to know how to implement them, and this way, with other Python classes, create custom deep learning models.

Until 15 of February

- Create the necessary infrastructure for research. A research pipeline in Google Colab Pro. Steps to do such:
    - Implement a simple RNN model with highly correlated synthetic data
    - From that, implement and test the 6 long-range area datasets.
    - Afterwards, with Optuna, optimize the RNN and create optimal LSTM, GRU, Mamba, and standalone CMS without integrating any mechanism and other baselines depending on what we test (like spiking neural networks), work with the benchmarks, and define all the metrics needed to know the quality of the models.
    - To guarantee gradient stabilization, use either residual blocks or the most recent variant, Manifold-Constrained Hyper-Connections (mHC), to ensure that the backpropagation algorithm works effectively.

    - See in long range area current benchmarks results as reference
- Get professor and TAs feedback in the process to know how to be better.

Until 5 of march

- Create our own nested Python classes to run our own new models with the continuous memory system and integrated deep learning models. First, start with simple RNNs, and from there, we make our own thing.
- Always save the results and compare them with baselines and current know benchmarks
- Get professor and TAs feedback in the process to know how to be better.

Until 15 of march

- Rigorous ablations studies on the best deep learning model to ensure rigor
- Use on interpretability (SHAP analysis or manifold) to understand how the model works
- Use monte Carlo dropout to understand, over many inferences runs, the variations of outputs and this way, the uncertainty of the model.
- Convert all the code developed into a codebase private GitHub repository with a good README file. This way we share the code and a link to the google collab notebook

Division of the work

Preethi work responsibilities - 33%

- ***Until 8 of February***
    - Understand all the mathematics of the continuous memory system concept in the paper: "Nested Learning: The Illusion of Deep Learning Architectures"
    - Study and understand the Long-Range area dataset benchmark based on the article: "Long Range Arena: A Benchmark for Efficient Transformers"
    - Understand how to use the Optuna Python library to guarantee optimal baselines and optimal custom model creation
    - Understand how to use PyTorch Lightning to know how to create new models easily.
    - Study Mamba, LSTM, GRU, xLSTM, and other gating mechanisms to understand how and why they were used. From the philosophy as to why they were used, we will create our own sequence models.
    - With PyTorch, learn to implement RNN, LSTM, and GRU to know how to implement them, and this way, with other Python classes, create custom deep learning models.
- ***Until 15 of February***
    - ***Get professor and TAs feedback in the process to know how to be better.***
    - Implement optimal LSTM in LRA benchmark in google colab notebook to serve as baseline.
    - Implement any other optimal deep learning in LRA benchmark in google colab notebook to serve as baseline. The more the better.
    - Implement any other benchmarks used in research in google colab notebook. The more the better.
- ***Until 5 of march***
    - Create our own nested Python classes to run our own new models with the continuous memory system and integrated deep learning models. First, start with simple RNNs, and from there we make our own thing.
        - In any new architecture, use residual blocks and manifold constrained hyper connections to make sure the backpropagation works well in all blocks of the model.
        - Explore the integration of continuous memory systems in mamba arquitecutre
    - Always save the results and compare them with baselines and current know benchmarks

- Get professor and TAs feedback in the process to know how to be better.
- Based on the results achieved, continue with ablation studies and others if everything is ok, or continue experimentation.

Nida work responsibilities - 33%

- ***Until 8 of February***
  - Understand all the mathematics of the continuous memory system concept in the paper: "Nested Learning: The Illusion of Deep Learning Architectures"
  - Study and understand the Long-Range area dataset benchmark based on the article: "Long Range Arena: A Benchmark for Efficient Transformers"
  - Understand how to use the Optuna Python library to guarantee optimal baselines and optimal custom model creation
  - Understand how to use PyTorch Lightning to know how to create new models easily.
  - Study Mamba, LSTM, GRU, xLSTM, and other gating mechanisms to understand how and why they were used. From the philosophy as to why they were used, we will create our own sequence models.
  - With PyTorch, learn to implement strach RNN, LSTM, and GRU to know how to implement them, and this way, with other Python classes, create custom deep learning models.
- ***Until 15 of February***
  - ***Get professor and TAs feedback in the process to know how to be better.***
  - Implement optimal GRU in LRA benchmark in google colab notebook to serve as baseline.
  - Implement any other optimal deep learning in LRA benchmark in google colab notebook to serve as baseline. The more the better.
  - Implement any other benchmarks used in research in google colab notebook. The more the better.
- ***Until 5 of march***
  - Create our own nested Python classes to run our own new models with the continuous memory system and integrated deep learning models. First, start with simple RNNs, and from there we make our own thing.
    - In any new architecture, use residual blocks and manifold constrained hyper connections to make sure the backpropagation works well in all blocks of the model.
  - Always save the results and compare them with baselines and current know benchmarks
  - Get professor and TAs feedback in the process to know how to be better.

- Based on the results achieved, continue with ablation studies and others if everything is ok, or continue experimentation.

Tiago's work responsibilities - 34%

Across all deadline, manage meetings and ensure group success

- ***Until 8 of February***
  - Understand all the mathematics of the continuous memory system concept in the paper: "Nested Learning: The Illusion of Deep Learning Architectures"
  - Study and understand the Long-Range area dataset benchmark based on the article: "Long Range Arena: A Benchmark for Efficient Transformers"
  - Study Mamba, LSTM, GRU, xLSTM, and other gating mechanisms to understand how and why they were used. From the philosophy as to why they were used, we will create our own sequence models.
  - Create a new google collab notebook with the long range arena benchmark already implemented from my previous experiments
  - Implement an optimal RNN model with optuna in the long range arena and a example for the other team members
- ***Until 15 of February***
  - Get professor and TAs feedback in the process to know how to be better.
  - Implement optimal continuous memory system according to how the paper defined it.
  - Implement any other benchmarks used in research in google colab notebook. The more the better.
- ***Until 5 of march***
  - Create our own nested Python classes to run our own new models with the continuous memory system and integrated deep learning models. First, start with simple RNNs, and from there we make our own thing.
    - In any new architecture, use residual blocks and manifold constrained hyper connections to make sure the backpropagation works well in all blocks of the model.
  - Always save the results and compare them with baselines and current know benchmarks.
  - Get professor and TAs feedback in the process to know how to be better.
  - Based on the results achieved, continue with ablation studies and others if everything is ok, or continue experimentation.

Conclusion

With first principles thinking, the project consists of using different components of deep learning systems and remixing them with CMS to seek new, better results on a common benchmark. We hope to achieve a great project in the machine learning class as well as a successful paper for the conference.

References

[1]

A. Vaswani *et al.*, "Attention Is All You Need," *Cornell University*, Jun. 12, 2017. https://arxiv.org/abs/1706.03762

[2]

A. Gu and T. Dao, "Mamba: Linear-Time Sequence Modeling with Selective State Spaces," *arXiv.org*, Dec. 01, 2023. https://arxiv.org/abs/2312.00752

[3]

M. Beck *et al.*, "xLSTM: Extended Long Short-Term Memory," *arXiv.org*, May 07, 2024. https://arxiv.org/abs/2405.04517

[4]

A. Behrouz, M. Razaviyayn, P. Zhong, and V. Mirrokni, "Nested Learning: The Illusion of Deep Learning Architectures," *arXiv.org*, 2025. https://arxiv.org/abs/2512.24695 (accessed Feb. 03, 2026).

[5]

K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," *arXiv.org*, Dec. 10, 2015. https://arxiv.org/abs/1512.03385